

Estimaciones en ambientes ágiles

 Si las estimaciones se utilizan como compromisos son muy peligrosas y perjudiciales para cualquier organización.

 Lo más beneficioso en las estimaciones es el “proceso de hacerlas”.

 La estimación podría servir como una gran respuesta temprana sobre si el trabajo planificado es factible o no.

 La estimación puede servir como una gran protección para el equipo.

Cualquier proceso de estimación trabaja en un universo de incertidumbre, con una probabilidad asociada. Lo contrario, es la certeza.

Lo que más incidencia tiene es saber cuál es el producto que el cliente quiere, contra lo que necesita.

Se confunde el concepto de estimación: ésta no es un compromiso. Se confunde el objetivo que quiero lograr, contra una estimación.

Una desventaja es que, cuando formulamos una estimación, deberíamos decir con que probabilidad le podemos acertar al valor que estamos prediciendo. Por ello, debe de venir asociada la probabilidad a que esa estimación se cumpla.

¿Cuándo algo es desperdicio? Es cuando no genera valor.

Las estimaciones nunca se negocian. Se negocian los compromisos.

Las estimaciones en ambientes ágiles son diferentes que las estimaciones tradicionales (se las verá en componentes de aspectos de SW).

Ninguna decisión se la irá a a tomar fuera de contexto. Uno debe ver el contexto y situación en cada momento y, en base a eso, revisar ⇒ a eso se refiere el “no ser dogmático sobre nada”.

Errores en las estimaciones.

1. Información imprecisa acerca del software a estimar o acerca de la capacidad para realizar el proyecto.
2. Demasiado caos en el proyecto (mal definido el proyecto).

3. Imprecisión generada por el proceso de estimación.
4. Una de las fuentes más común es omitir actividades necesarias para la estimación del proyecto tales como, requerimientos faltantes, licencias, reuniones, revisiones, etc.
5. Se olvidan del detalle que planear no es estimar y estimar no es planear, sino que las estimaciones son la base de los planes, pero los planes no tienen que ser lo mismo que lo estimado.
6. A mayor diferencia entre lo planeado y lo estimado, mayor el riesgo.
7. Las estimaciones no son compromisos.

Matriz de desambiguación

Objetivo: una meta de negocio deseada.

Nosotros trabajamos para alguien/algo, y ese "algo" es la meta de negocio a alcanzar, para lo cual el SW a elaborar va a colaborar; a eso se refiere con objetivo (target). Un objetivo de negocio está elaborado por alguien, por lo que es subjetivo.

Estimación: Una predicción analítica o imparcial del esfuerzo o costo.

Estimar es PREDECIR. Al dar una estimación, se trata de ser lo mas objetivo posible, y la objetividad viene en base a que usamos parámetros y observaciones que sean lo más sistematizadas posibles.

Estimar es predecir una ocasión que ocurrirá a futuro.

Compromiso: una promesa de entrega asumiendo riesgos específicos.

El compromiso es lo que se acuerda después con el cliente, una vez que se tiene la estimación. Es lo que esperamos lograr con la planificación. La planificación toma como entrada una estimación, y logra establecer con el cliente un compromiso.

La dinámica de los procesos de SW

Una estimación artificialmente baja no acelera el desarrollo; recorta las fundaciones, asegurando que los errores se descubran en la fase más cara del proyecto.

1. Cronograma irreal dictado por el negocio: a veces se comete el error de un cronograma irreal dictado por el negocio, que es el punto de la partida de la mayoría de ellos.
2. Se recortan actividades tempranas (análisis, diseño): como se partió de un cronograma de trabajo subestimado, como consecuencia, uno tiene que mal actuar, por lo que se recortan actividades tempranas que deberían de ser necesarias hacer porque, al mediano plazo, permite recortar gastos que no plantearlo.
3. Se programan soluciones rápidas y sucias: se plantean malas soluciones y, como consecuencia, se tienen:

4. Altas tasas de defectos en pruebas o producción.
5. Reuniones de crisis, refactorización tardía y desvío de personal.

Las estimaciones tempranas son erróneas. No podemos planificar para eliminar la incertidumbre, sino iterar para descubrirla. La certidumbre se logra con SW funcionando.

Las estimaciones se piden en un tiempo demasiado corto, eso lo que plantea el "cono de la incertidumbre". Al estimar al inicio del proyecto, no tenemos la info. suficiente para hacer una estimación.

El agilismo propone que la certeza va a venir con el feedback del SW funcional. El cono se achica porque, a medida que llegamos al final del proyecto, la incertidumbre va desapareciendo.

Sobreestimar: decir que necesito más tiempo del que realmente necesito.

¿Cómo se hace/logra una buena estimación?

1. Debe estar expresada en rangos. Es para expresar cierto margen que puede aceptar el cliente. Ej.: 12-16 semanas.
2. Incluye un porcentaje de probabilidad explícito. Es el informe del porcentaje de probabilidad.
3. Totalmente separada del objetivo de negocio.
4. Habilita decisiones de control.

Tamaño / Estimaciones Relativas y Absolutas

Las estimaciones ágiles se trabajan con estimaciones relativas, no absolutas. Significa que las personas somos mejores estimando por comparación: "esto es más complejo que esto...".

La absoluta, refiere a dar un valor: "necesito 4 horas", "esto es muy complejo y tiene este tamaño". El agilismo invita a abandonar esto. La estimación absoluta hace que sea más fácil equivocarnos.

La complejidad del software crece exponencialmente. Cuando vamos a elegir una escala de medición, elegiremos la de Fibonacci para reflejar que, a mayor tamaño, mayor es la incertidumbre.

Las features/stories son estimadas usando una medida de tamaño relativo conocida como story points (SP). No es una medida basada en tiempo. Las SP son la unidad de medida para estimar las user stories.

Lo que vamos a estimar de la user, es su tamaño.

Entonces, hacemos énfasis en considerar, antes de plantear fechas/costos, las US y su tamaño. El tamaño refiere al producto a construir, mientras que el esfuerzo se mide

siempre en horas persona lineales.

En el enfoque ágil, el esfuerzo es una cosa individual, la cual no se puede sistematizar. El agilismo dice que, quien debe estimar, es quien hace el trabajo. A diferencia del enfoque tradicional, quien estima es el líder de proyecto, quien estima desde su lugar.

Formas de estimar el tamaño

Existen diferentes escalas para estimar el tamaño y siempre son por comparación, entre ellas:

1. Número del 1 al 10.
2. Talles de remeras: S, M, L: suele ser utilizado como escala de estimación para los ítems del Product Backlog.
3. Serie exponencial de base 2: 2, 4, 8, 16, 32, 64, etc.
4. Serie de Fibonacci: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, etc. El número para estimar la US debe estar en la escala de Fibonacci. La serie de Fibonacci es la escala más utilizada en metodologías ágiles para estimar el tamaño y la complejidad de las Historias de Usuario (Story Points), principalmente a través de la dinámica de Planning Poker.

En la práctica se suele usar la Serie de Fibonacci Modificada (popularizada por Mike Cohn):

0, 0.5, 1, 2, 3, 5, 8, 13, 20, 40, 100

(Nota: A partir del 13, se redondea a 20, 40 y 100 para evitar dar una falsa sensación de precisión, ya que en la Fibonacci clásica seguirían 21, 34, 55, 89)

Una vez definida la escala, no se cambia. En caso de cambiarla, también se debe cambiar el patrón de medición

¿Qué es un story point?

Es una unidad de estimación que especifica la complejidad, incertidumbre y esfuerzo propio del equipo respecto a una US en términos relativos respecto a otra US. Decide cuán grande/compleja es. Por lo general, esa complejidad tiende a incrementarse exponencialmente (debido a la sucesión de Fibonacci).

Tiene 3 dimensiones: complejidad, el esfuerzo (de quien hace el trabajo, porque es ambiente ágil) e incertidumbre (o duda).

1. Complejidad: que tan compleja es implementar esa US.
2. Esfuerzo: son las horas persona lineales que se requieren para terminar la US.

Las horas-persona lineales son la estimación o cálculo de esfuerzo bajo el supuesto de que el tiempo de entrega de un trabajo es directamente proporcional a la cantidad de personas asignadas, asumiendo que cada hora invertida produce siempre el mismo rendimiento constante y que el trabajo se puede dividir de manera perfecta.

3. Incertidumbre: es cuánta información falta para ser más adecuados en la estimación.

Velocidad

La velocidad es la cantidad de puntos de historia que el PO me aceptó al final de la iteración. La velocidad NO se estima, se CALCULA. Se calcula sumando y multiplicando.

- Es una medida (métrica) del progreso de un equipo.
- Se calcula sumando el número de story points (asignados a cada user story) que el equipo completa durante la iteración.
- Se cuentan los story points de las Users Storys que están completas, no parcialmente completas.
- La velocidad corrige los errores de estimación.
- La velocidad cuenta la cantidad de SP que me aceptó el PO cuando presenté el incremento del producto.

Si una US1 = 3sp, US2 = 5sp y US3 = 1sp y el PO aceptó la US1, se suman 3 sp de velocidad.

Poker estimation / Poker planning

=====

LEER

1. ¿Qué es el Planning Poker?

El Planning Poker (o Scrum Poker) es una técnica ágil, colaborativa y gamificada (basada en el juego) que utilizan los equipos de desarrollo para estimar el tamaño, complejidad y esfuerzo de las Historias de Usuario (User Stories).

En lugar de estimar en horas fijas o lineales, el equipo estima en Story Points (Puntos de Historia), una unidad de medida relativa que combina tres factores:

- Complejidad técnica: Qué tan difícil o intrincada es la solución.
- Volumen de esfuerzo: La cantidad de trabajo a realizar.
- Incertidumbre y riesgo: Lo desconocido, dependencias externas o falta de claridad tecnológica.

2. La Escala de Estimación: Serie de Fibonacci Modificada

Durante la dinámica, cada participante cuenta con un mazo de cartas con valores basados en la Serie de Fibonacci Modificada:

0, 0.5, 1, 2, 3, 5, 8, 13, 20, 40, 100

(Incluye además cartas especiales como el signo ? para expresar desconocimiento total y la taza de café ☕ para solicitar un descanso).

3. La Dinámica Paso a Paso

1. Presentación de la Historia: El Product Owner (PO) lee la Historia de Usuario, define el objetivo de negocio y detalla los Criterios de Aceptación.
2. Aclaración de dudas: El equipo de desarrollo realiza preguntas breves sobre la arquitectura, diseño o dependencias para alinear expectativas.
3. Votación a ciegas (Secreta): Cada miembro del equipo elige una carta de su mazo y la coloca boca abajo. Nadie puede decir números en voz alta ni hacer comentarios antes de votar.
4. Revelación simultánea: A la cuenta de tres, todos los integrantes muestran su carta al mismo tiempo.
5. Debate de los extremos:
 - Si la mayoría coincide (ej. todos 3 o 5), se toma ese valor como consenso final.
 - Si hay divergencia notable (ej. alguien votó 2 y otro votó 13), solo argumentan los extremos:
 - El voto más bajo: Explica si conoce una forma más simple, una librería existente o una solución rápida que los demás no vieron.
 - El voto más alto: Explica qué riesgos ocultos, complejidades arquitectónicas o casos de borde identificó.
6. Re-votación y Consenso: Tras un breve intercambio (2-3 minutos), el equipo vuelve a votar en secreto. Por lo general, en la segunda ronda se alcanza el acuerdo de manera natural.

4. ¿Por qué es una herramienta tan potente?

- Elimina el Sesgo de Anclaje y la Autoridad: Al votar en secreto, se evita que los desarrolladores más senior o líderes técnicos influyan inadvertidamente en el criterio de los integrantes con menos experiencia.

- Fomenta el Entendimiento Compartido: El mayor valor del Planning Poker no es el número asignado, sino la conversación técnica que surge al comparar los extremos. Permite descubrir problemas y alinear la arquitectura antes de escribir una sola línea de código.
- Rapidez y Enfoque: Convierte una sesión de estimación larga y tediosa en una actividad dinámica, participativa y orientada a la toma de decisiones ágiles

=====

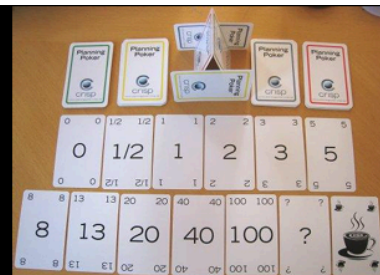
Poker Planning

Prerrequisitos:

- Lista de features/stories a ser estimadas
- Cada estimador tiene un mazo de cartas.

Pasos:

1. Determine la **base story** (la canónica) que será usada para comprar con las otras stories. Digamos, Story Z.
 - 1.1 La story a ser estimada se lee a todo el equipo.
 - 1.2 Los estimadores discuten la story, haciendo preguntas al product owner (las que se necesiten).
 - 1.3 Cada estimador selecciona una carta y pone la carta boca abajo en la mesa.
 - 1.4 Cuando todos pusieron las cartas, las mismas se exponen al mismo tiempo.
 - 1.5 Si todos los estimadores selecciona el mismo valor, ese es el estimado. Sino, los estimadores discuten sus resultados, poniendo especial atención en los más altos y los más bajos. Después de la charla, GOTO to 1.3.
2. Se toma la próxima story, se discute con el product owner.



3. Cada estimador asigna a la story un valor por comparación contra la base story.
 "Cuan grande/pequeña,
 compleja, riesgosa es esta story comparada con Story Z?"

Se elige una user que para el equipo signifique lo mas simple, y a eso se le suma 1 sp y se la pone como "canonica". Todas las historias se comparan contra esa y decimos "esta historia es 'x' veces más difícil que esta otra".

Se deja el 1/2 como un margen para decir que apareció algo que es más sencillo.

El 0 se usa para RNF. Si hay mucha incertidumbre, "?". Si nos cansamos, una carta de café. Hoy casi no se usan cartas debido a que los trabajos son remotos.

0	1/2	1	2
3	5	8	13
20	40	100	?

- **0**: Quizás ud. no tenga idea de su producto o funcionalidad en este punto.
- **1/2, 1**: funcionalidad pequeña (usualmente cosmética).
- **2-3**: funcionalidad pequeña a mediana. Es lo que queremos. 😊
- **5**: Funcionalidad media. Es lo que queremos 😊
- **8**: Funcionalidad grande, de todas formas lo podemos hacer, pero hay que preguntarse sino se puede partir o dividir en algo más pequeño. No es lo mejor, pero todavía 😊
- **13**: Alguien puede explicar por que no lo podemos dividir?
- **20**:Cuál es la razón de negocio que justifica semejante story y más fuerte aún, por qué no se puede dividir?.
- **40**: no hay forma de hacer esto en un sprint.

100: confirmación de que algo está muy mal. Mejor ni arrancar.

Como vamos a trabajar con el concepto de "just in time", vamos a estimar lo que trabajemos en este mismo sprint.

Para el sprint backlog, usar Fibonacci. Para los items de product backlog, la técnica de talle de remera.

¿Cuándo debemos re estimar?

Cuando me di cuenta como equipo que esto es más complejo a como lo pensamos. Re estimamos el tamaño de la user. Si el equipo determina que el tamaño no era el indicado, se reestima.

ECUACION CENTRAL ÁGIL

En ágil, nunca estimamos el tiempo directamente. Estimamos la magnitud del trabajo, medimos nuestra velocidad empirica y calculamos el cronograma:

$TAMAÑO (SP) / VELOCIDAD (puntos por iteración) = Duración (iteraciones)$

Las iteraciones en ágil son de duración fija. La fecha de fin, no se mueve. Se estima el tamaño, se deriva la duración.

Se deben de tener en cuenta todas las características de la user.